

Unidad 2

Estrategias de división de servicios

**En este capítulo vamos a estar
respondiéndonos las preguntas...**

**¿Cómo dividir un servicio de
otro? ¿Cómo definir sus
fronteras?**

Parte importante de lo que veremos en este capítulo son técnicas y criterios utilizados para dividir módulos. Como microservicios es un tipo de arquitectura modular, veremos que gran parte aplica a esta arquitectura.

Definir las fronteras de microservicios utilizando los criterios de [Parnas, 1972]:

- Información oculta
- Cohesión
- Acoplamiento

Información oculta

"La conexión entre los módulos son las suposiciones que deben hacer los módulos entre ellos" [Parnas, 1972]. Hay que reducir la conexión entre los servicios.

Esto significa mejor...

- **Velocidad de desarrollo:** se pueden agregar desarrolladores en paralelo.
- **Comprensión:** se puede entender cada parte sin entender el total.
- **Flexibilidad:** se puede cambiar una parte sin tocar el resto.

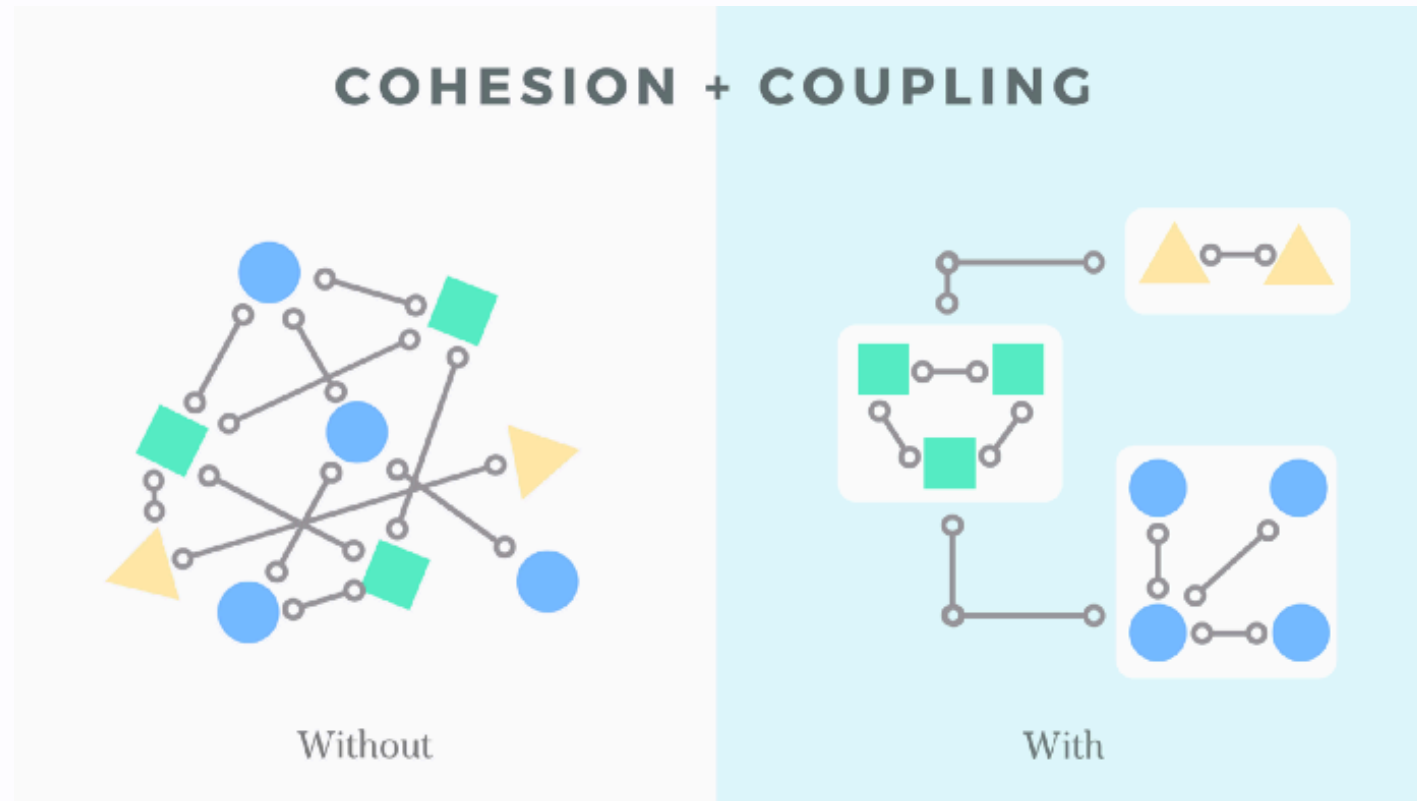
Cohesión

"Mantén junto todo lo que cambia por la misma razón..." (Robert C. Martin)

Acoplamiento

Un servicio no debe requerir que otro servicio cambie para poder cambiar él.

Constantine's Law: "Una estructura es estable si tiene alta cohesión y bajo acoplamiento."





Tipos de acoplamiento

Dominio ↔ Paso ↔ Común ↔ Contenido

Menos grave ↔ ... ↔ Más grave

Acoplamiento de dominio

Esto se da cuando un servicio requiere interactuar con otro servicio para utilizar una funcionalidad que este tiene. Es inevitable en grandes sistemas pero **debe ser minimizada**.

Ejemplo común: se requiere confirmar si el usuario tiene permisos para realizar la acción solicitada: se consulta a un servicio de autenticación.



Acoplamiento de paso

Esto sucede cuando en una llamada de un servicio a otro, se envían datos para que este llame a un tercero.

Un ejemplo es cuando el servicio de compras le envía información adicional al servicio de inventario para que este pueda llamar al servicio de 'delivery' (shipping).



Acoplamiento común

Cuando dos o más servicios comparten la misma fuente de datos o estado: la misma tabla, la misma base de datos, el mismo archivo. No importa qué atributos escriba cada uno.

Cualquier cambio a la estructura de esos datos obliga a coordinar a todos los servicios que la usan. Además, un servicio puede dejar los datos en un estado que rompe las suposiciones de los demás.

Debe evitarse.

Acoplamiento de contenido

Cuando un servicio alcanza los internos de otro y los modifica saltándose su frontera. El ejemplo típico: escribir directo en la base de datos de otro servicio en vez de usar su API.

La frontera del servicio deja de existir: su dueño ya no puede garantizar sus propias reglas ni cambiar su esquema sin romper al intruso. En caso de error, es difícil trazar qué servicio realizó el cambio. **Debe evitarse a toda costa.**

También se le conoce como **acoplamiento**

Domain Driven Design (DDD)

Es una propuesta de Eric Evans (2003) para representar de mejor manera un sistema en el contexto en el que se desenvuelve.



Lenguaje ubicuo

Definir y adoptar un lenguaje común para el diseño del sistema, el código y el negocio. Tener un lenguaje ubicuo permite ahorrar tiempo y disminuir el roce en la toma de requerimientos.



Agregación

Un conjunto de objetos que son manejados como una sola entidad, habitualmente, refiriéndose a un concepto del mundo real.

Un servicio está a cargo del ciclo de vida de una o más agregaciones.

Raíz de agregado e invariantes

Cada agregación tiene una **raíz**: la entidad por la que se accede a todo el resto. Nadie modifica un objeto interno directamente, todo pasa por la raíz.

¿Por qué? Porque la agregación existe para proteger **invariantes**: reglas del negocio que deben cumplirse siempre. Ejemplo: "un paralelo no puede superar sus cupos". Si la inscripción y los cupos viven en la misma agregación, la raíz puede garantizar esa regla en cada operación.

La invariante define la frontera: lo que debe ser

Las agregaciones se referencian entre sí **por su identidad** (su ID), no cargando el objeto completo.

Dentro del mismo servicio, esa referencia puede terminar implementada como una clave foránea, pero eso es un detalle de la base de datos relacional, no del modelo.

Cuando la relación es con una agregación de otro servicio, dependencia entre-servicios, se utiliza la ID remota o su URI (*Uniform Resource Identifier*).

Contexto acotado

Fronteras explícitas alrededor de una parte del dominio del negocio que entrega una funcionalidad al sistema al mismo tiempo que oculta su complejidad.

Pueden contener una o más agregaciones. Algunas se exponen (*shared models*) y otras se ocultan (*hidden models*).



Ejemplo: gestión académica universitaria

Tomemos el sistema que usaremos en la unidad 3: estudiantes, profesores, cursos, inscripciones, calificaciones y aranceles.

Un primer corte en contextos acotados: **Usuarios, Cursos, Calificaciones y Aranceles**. Cada uno tiene su propio lenguaje y sus propias reglas.

Notemos que "estudiante" significa algo distinto en cada contexto:

- En **Usuarios**: una persona con credenciales y datos personales.
- En **Cursos**: alguien que ocupa un cupo en un paralelo.
- En **Aranceles**: un deudor con pagos y beneficios.

No hay una única clase Estudiante compartida. Cada contexto tiene su propio modelo y comparten solo la identidad (el ID del estudiante). Eso es información oculta aplicada al dominio.

Agregaciones dentro del contexto **Cursos**: Curso, Paralelo e Inscripción.

La raíz Paralelo protege la invariante de cupos: inscribir un estudiante pasa por el paralelo, que verifica cupos disponibles antes de aceptar.

La Inscripción referencia al estudiante por su ID remota: el contexto Cursos no conoce (ni le importa) la contraseña o el RUT del estudiante. Eso vive en Usuarios.



Event Storming

Técnica de taller propuesta por Alberto Brandolini para descubrir el dominio junto a los expertos del negocio.

En una pared (o un Miro) se colocan los **eventos del dominio** en orden temporal: "estudiante inscrito", "arancel pagado", "calificación publicada". Luego se agregan los comandos que los gatillan y las agregaciones que los procesan.

Los grupos de eventos que quedan juntos, y los cambios de lenguaje entre grupos, sugieren dónde



Ley de Conway

"Las organizaciones que diseñan sistemas están condenadas a producir diseños que son copias de sus estructuras de comunicación." (Melvin Conway, 1968)

Si dos equipos deben coordinarse para cada cambio, sus servicios terminarán acoplados. La estructura de equipos y la arquitectura se moldean mutuamente: conviene diseñarlas juntas.



Equipos stream-aligned

Team Topologies (Skelton y Pais, 2019) propone organizar el desarrollo en torno a equipos **stream-aligned**: alineados a un flujo de valor del negocio, dueños de sus servicios de punta a punta.

Esto conecta con la Two-Pizza Rule que vimos en la unidad 1: un equipo chico, con autonomía, que no necesita pedir permiso a otro equipo para desplegar. Las fronteras de los servicios deberían coincidir con las fronteras de los equipos.



¿Descomponer por subdominio o por capacidad de negocio?

- **Por subdominio** (DDD): se divide según el modelo del dominio y sus contextos acotados. Mira los datos y las reglas: Cursos, Aranceles.
 - **Por capacidad de negocio**: se divide según lo que la organización hace para generar valor. Mira los procesos: inscribir estudiantes, cobrar aranceles.
- En la práctica suelen converger. Cuando difieren, la capacidad de negocio manda: es más estable en el tiempo que el modelo de datos, y es lo que un



Dividiendo el monolito



Migración incremental

"Si haces una reescritura a lo big-bang, lo único de lo que estás garantizado es una gran explosión."

(Martin Fowler)

Una estrategia incremental permite limitar el impacto de errores-problemas, ir aprendiendo en el proceso sobre cómo construir microservicios y permite recibir los beneficios a medida que se vaya construyendo.



Coexistencia entre el monolito y los microservicios

Una arquitectura monolítica **no** es intrínsecamente mala. El foco al migrar no es “no tener un monolito”. El foco son los beneficios.

Es común que un monolito coexista con microservicios, a menudo con capacidad disminuida.

Peligros de la descomposición prematura

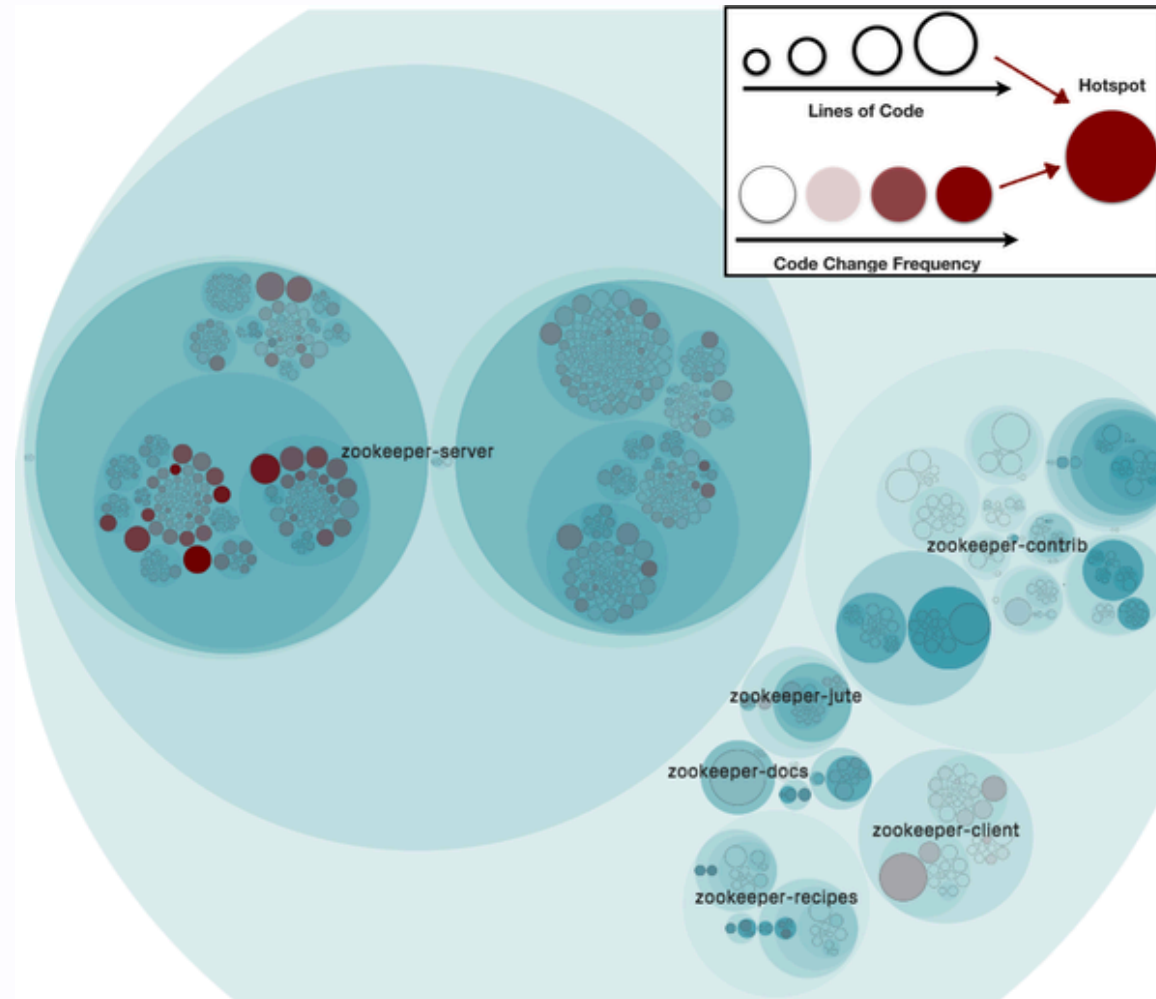
Existe peligro en la implementación de microservicios cuando se tiene una comprensión poco clara del dominio.

¿Qué dividir primero?

Se debe comenzar por las partes que más se beneficien de la división, eso depende de los objetivos.

¿Se quiere escalar? ¿Se quiere mejorar el time-to-market? ¿Qué tan viable es separar la funcionalidad?

Análisis de código con CodeScene de Apache Zookeeper



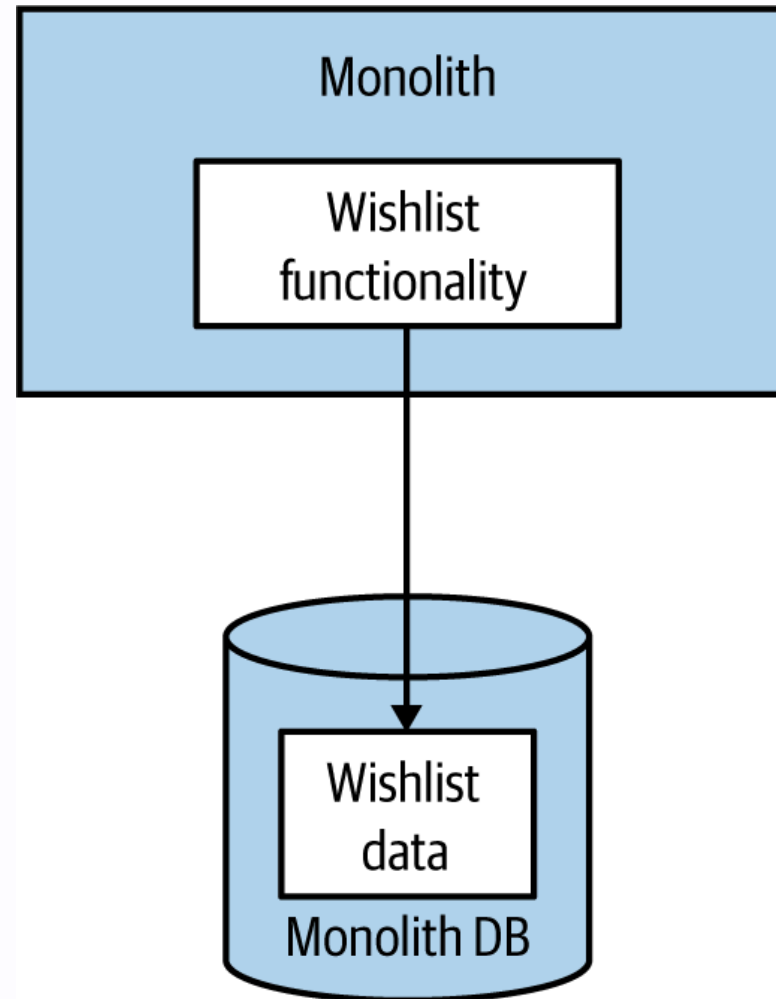
La decisión sobre qué dividir es un equilibrio entre **lo fácil** que es la extracción y **el beneficio** de extraer. Costo beneficio.



Descomposición por capa

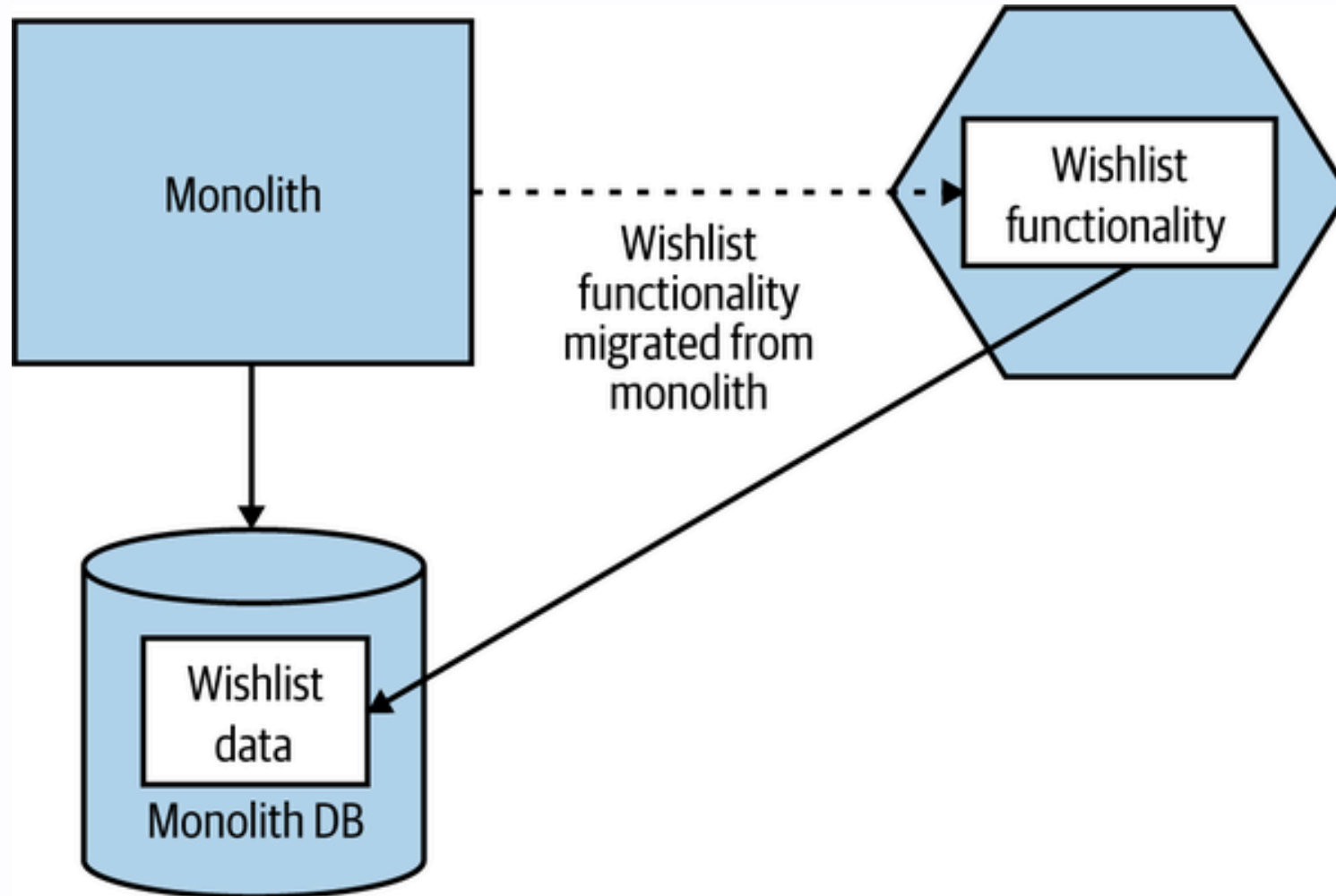
Al desacoplar un sistema monolítico, se debe considerar tanto el código (backend) como los datos (base de datos).

El código y los datos que se desea extraer del monolito.





Código primero

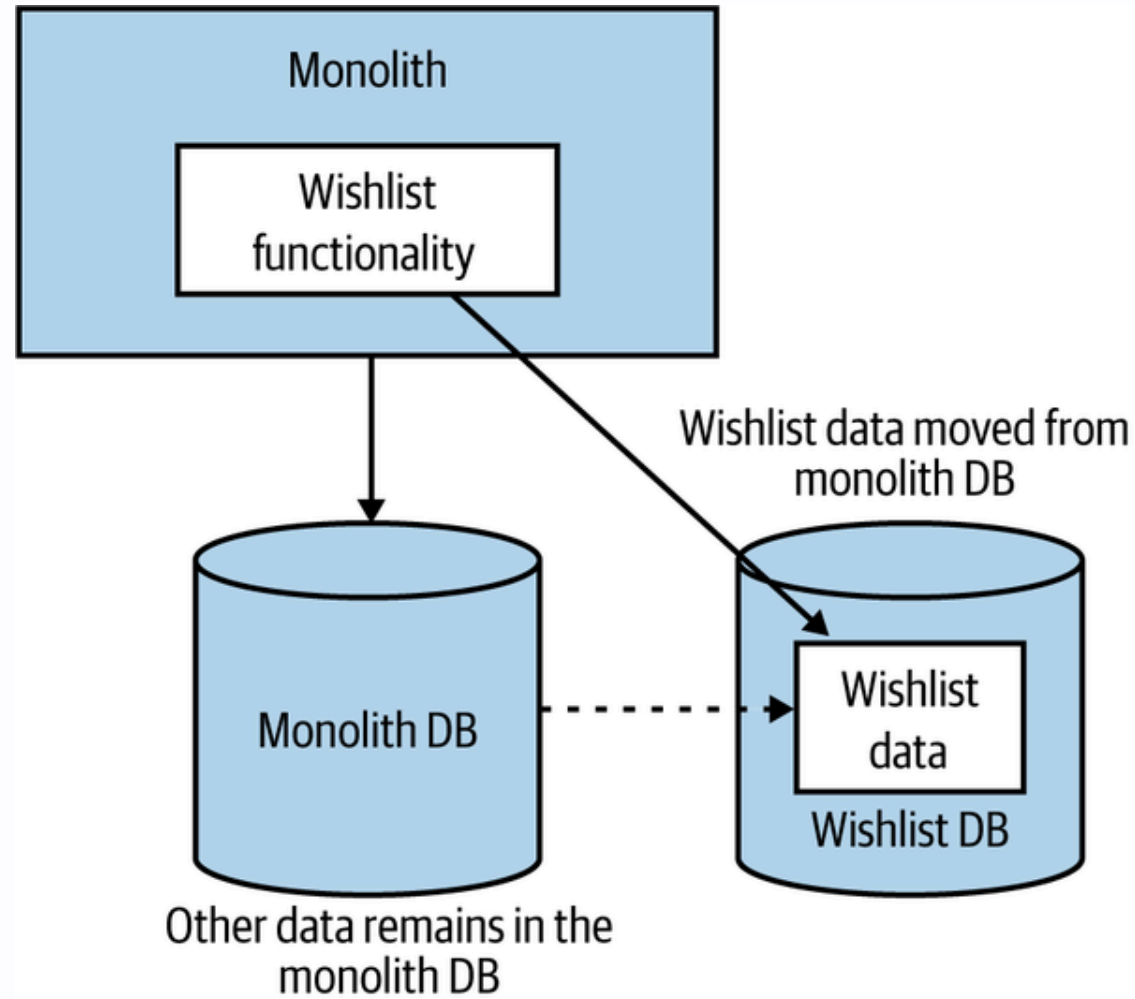


Se extrae el código asociado con la funcionalidad en un nuevo microservicio. Los datos permanecen en la base de datos monolítica en esta etapa.

No hemos completado la descomposición hasta que también hemos movido los datos relacionados con el nuevo microservicio.

Tiende a ser más fácil extraer el código que los datos pero existe el riesgo de postergar la extracción de los datos (y mantener el acoplamiento).

Datos primero



Se extraen los datos de la base de datos asociados con la funcionalidad primero. El código permanece en el monolito en esta etapa.

A pesar de ser una estrategia más compleja, reduce el riesgo de postergar la extracción de los datos.

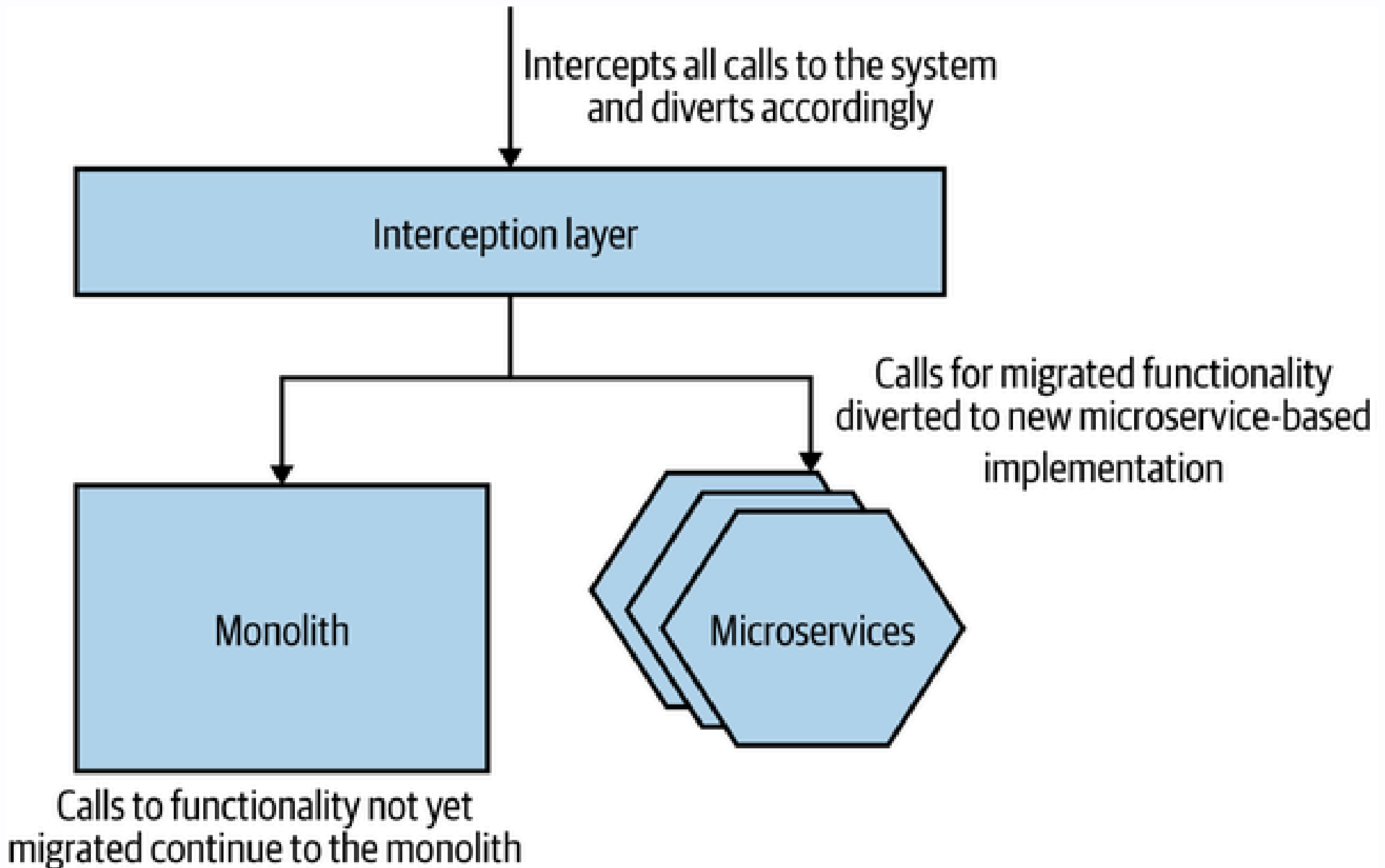
Patrones de descomposición



Patrón *Strangler Fig*

Envolver un sistema antiguo con el nuevo sistema a lo largo del tiempo, lo que permite que el nuevo sistema se haga progresivamente.

En este contexto, se implementa un servicio que intercepta todas las llamadas al sistema. La llamada se redirige a algún microservicio o al monolito, dependiendo de quién la implemente.



👁️ Correr en paralelo

Al cambiar de arquitectura, en un sistema bien probado a uno nuevo, puede existir preocupación. Una forma de mitigar esto es ejecutar tanto el monolito como los nuevos microservicios en paralelo, respondiendo las mismas consultas y comparando los resultados.

● *Feature Toggle*

Feature Toggle es un patrón que permite activar o desactivar una funcionalidad o cambiar entre dos implementaciones de una funcionalidad en tiempo de ejecución.

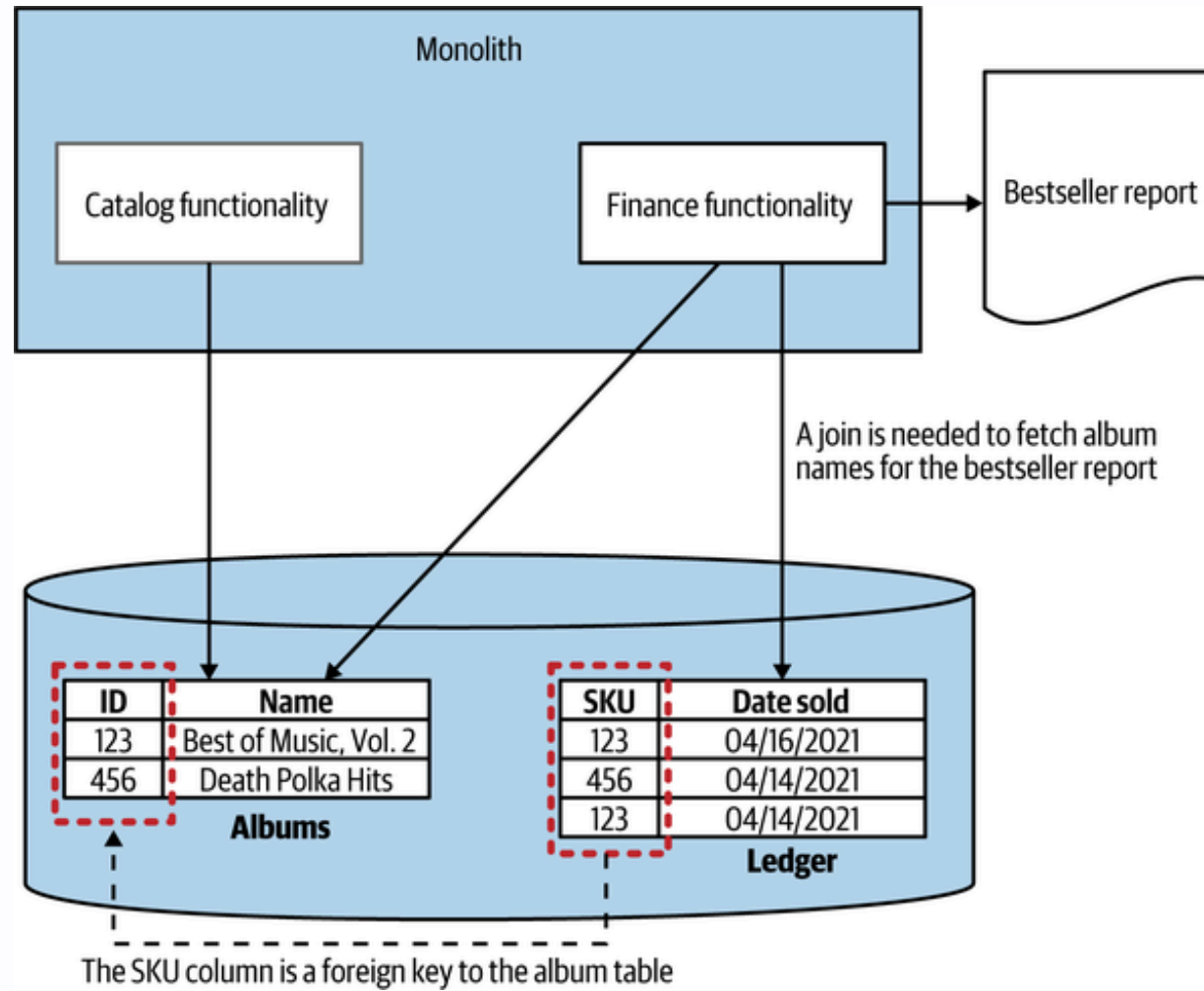
En este contexto, podemos implementar *feature toggle* en el *proxy* para controlar a qué usuarios se les entrega qué implementación.

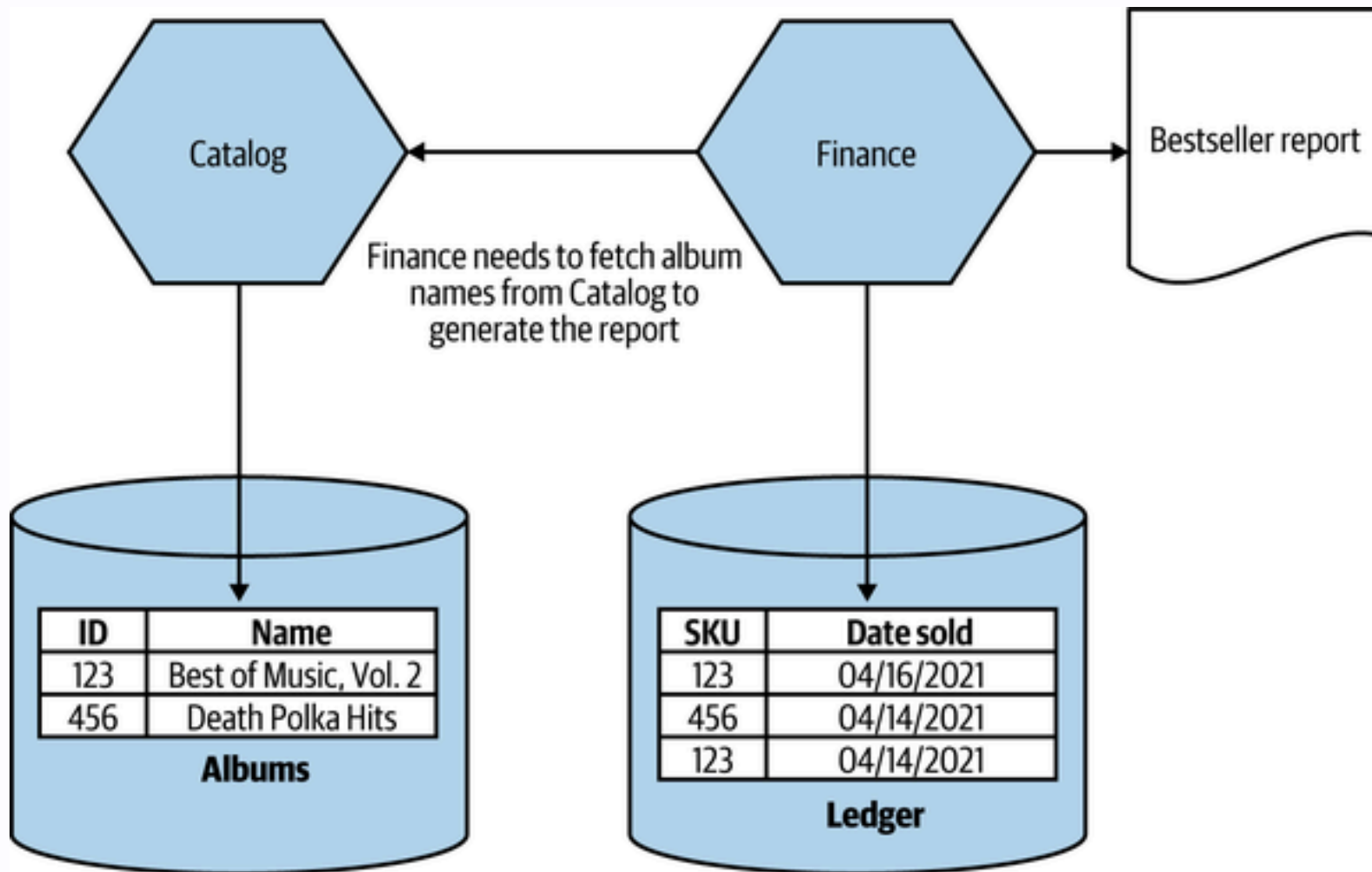
Preocupaciones al descomponer los datos

Rendimiento

Las bases de datos son buenas para unir (*join*) datos.

Al separar los datos en múltiples microservicios, algunas uniones (*joins*) pasan de la capa de datos (*data tier*) al código de la aplicación (*code tier*).





Algunas estrategias para mitigar esto pueden ser permitir operaciones *bulk* por la variable por la que se va a realizar la unión o algún sistema de caché.



Integridad de los datos

Con tablas en diferentes bases de datos, **hay verificaciones que no pueden vivir en el mismo modelo.**

Por ejemplo, no podemos incluir una llave foránea, verificando, antes de borrar una entidad, que no existen entidades relacionadas.

No **podemos confiar** en la base de datos para mantener la integridad de los datos.

Existen algunas estrategias que pueden ayudar, cada una con sus costos y beneficios que se deben evaluar en el contexto del proyecto: realizar ***soft delete*** de tablas que sean referenciadas por otros servicios, tener **datos duplicados**, realizar **transacciones entre servicios**, entre otras.

Transacciones

Al dividir los datos en múltiples bases de datos, **perdemos la atomicidad entre servicios**: no hay una transacción que abarque las dos bases. Cada base de datos sigue siendo ACID a nivel local.

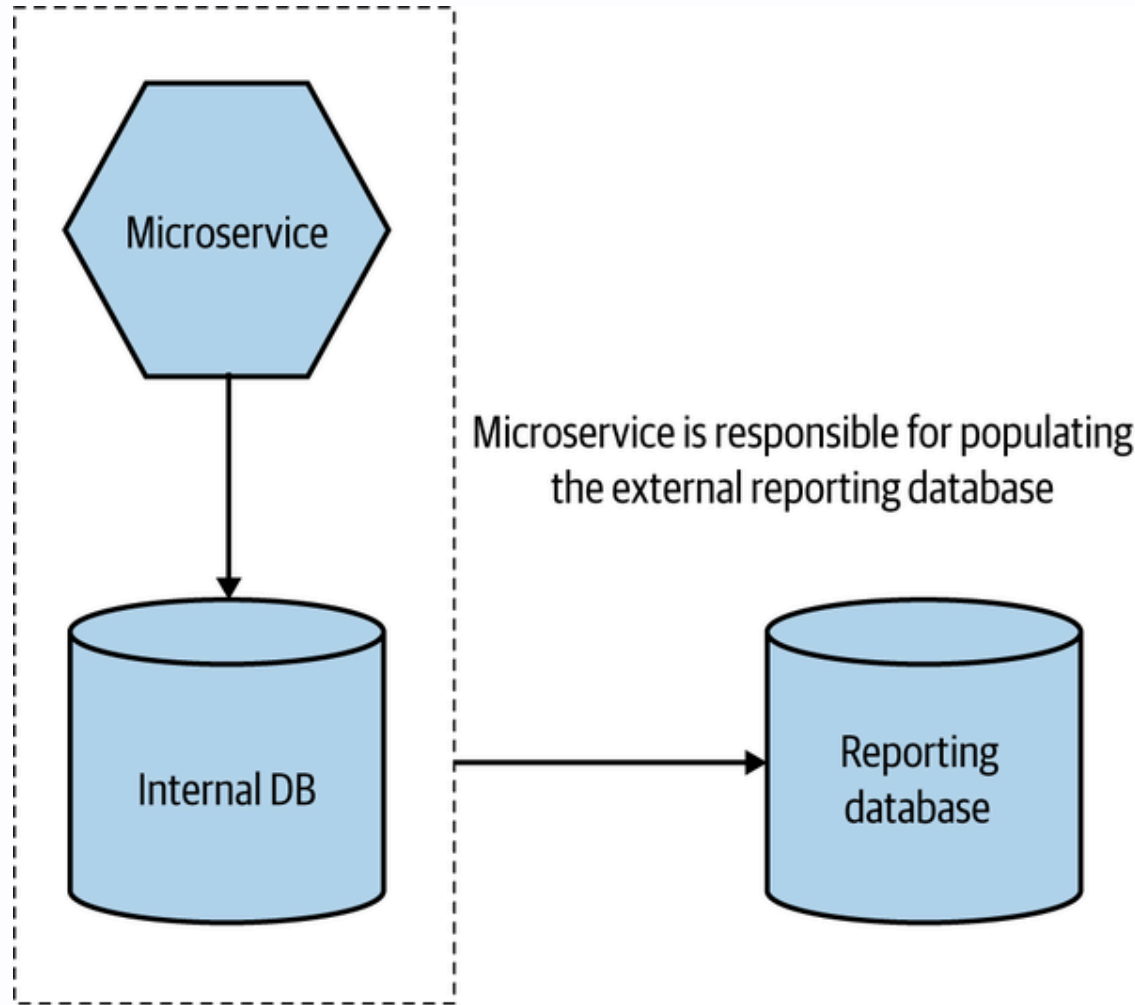
Una operación de negocio que toca varios servicios puede quedar a medias. Cómo manejar esto (sagas) lo abordaremos en más detalle en la Unidad 4.



Base de datos de informes

Al dividir nuestra base de datos, también debemos permitir solo a su servicio acceder a dichos datos. Esto permite crear interfaces estables, evitar errores y disminuir problemas de integridad.

Hay casos donde se requiere información de distintos servicios juntas, habitualmente para generar reportes.



Una estrategia es crear una base de datos de informes, que contiene datos de múltiples servicios. Estos pueden poblar esta base de datos directamente o mediante una API REST. Esto se puede hacer en tiempo real o en un proceso *batch*. Esto permite que otro servicio pueda acceder a los datos de diversos servicios.



Material complementario

- Building microservices: Designing fine-grained systems, Sam Newman (2021). O'Reilly. Capítulos 2 y 3.
- On the Criteria To Be Used in Decomposing Systems into Modules, [Parnas, 1972](#).
- On the criteria to be used in decomposing systems into modules, [Colyer, 2016](#).

- A Handbook of Software and Systems Engineering, Albert Endres and Dieter Rombach (2003).
- Domain-Driven Design: Tackling Complexity in the Heart of Software, Eric Evans (2003).
- StranglerFigApplication, Martin Fowler (2004).
<https://martinfowler.com/bliki/StranglerFigApplication.html>

- Feature Toggles (aka Feature Flags), Martin Fowler (2017).
<https://www.martinfowler.com/articles/feature-toggles.html>
- Sam Newman, Monolith to Microservices (2019), O'Reilly.